

Implementation Work: Metaheuristics

Nicolas Von Dolinger Moreira Rocha

2022035571

Federal University of Minas Gerais - UFMG

October 19, 2025

Abstract

The Traveling Salesperson Problem (TSP) is a fundamental NP-hard challenge in combinatorial optimization, requiring the determination of the shortest possible route that visits a set of cities and returns to the origin. This work focuses on the implementation and performance analysis of a metaheuristic to generate high-quality approximate solutions for the TSP. The chosen method is the **Greedy Randomized Adaptive Search Procedure (GRASP)**. This approach consists of two main phases: a constructive phase, which builds an initial solution using a guided, randomized greedy strategy, and a local search phase, which refines this solution to a local optimum. For this implementation, the **2-opt** algorithm was used as the local search component. The entire process is executed iteratively, and the best tour found across all iterations is selected as the final solution.

1 Algorithm

The metaheuristic chosen for this work is the **Greedy Randomized Adaptive Search Procedure (GRASP)**. GRASP is a multi-start method where each iteration consists of two phases: construction and local search.

1.1 Phase 1: Solution Construction

The first phase is the **Constructive Phase**, which builds a feasible solution incrementally. The process begins at a randomly selected starting node. In each subsequent step, the algorithm constructs a **Restricted Candidate List (RCL)** of the best next nodes to visit (i.e., unvisited nodes that are "close" to the current node).

The "adaptiveness" of GRASP comes from how this RCL is built. It is not purely greedy (i.e., only selecting the single best node). Instead, it uses a parameter α (where $0 \leq \alpha \leq 1$) to define a quality threshold. This threshold is calculated based on the minimum cost (c_{min}) and maximum cost (c_{max}) of all possible next moves:

$$\text{limit} = c_{min} + \alpha \times (c_{max} - c_{min})$$

All unvisited nodes whose connection cost is less than or equal to this limit are added to the RCL. A node is then chosen *at random* from this RCL to be the next node in the tour. This process is repeated until all nodes have been visited, at which point the tour

is completed by returning to the starting node. This α parameter balances greediness (a small α creates a small, high-quality RCL) and randomness (a large α allows for more diverse, exploratory choices).

1.2 Phase 2: Local Search

The tour generated by the constructive phase, while likely good, is almost certainly not a local optimum. Therefore, it is passed to a **Local Search** phase to be improved.

For this work, the **2-opt** algorithm was implemented. This algorithm iteratively improves a tour by searching for pairs of non-adjacent edges that can be swapped to create a shorter overall route. Specifically, it tests swapping edges (u_1, v_1) and (u_2, v_2) with (u_1, u_2) and (v_1, v_2) . If the new path is shorter, the swap is made by reversing the sequence of nodes between v_1 and u_2 . This process is repeated until no further 2-opt improvements can be found, at which point the tour is considered "2-optimal".

The entire GRASP process (Phase 1 + Phase 2) is repeated for a fixed number of iterations, and the best tour found across all iterations is saved as the final solution.

2 Results

The performance of the GRASP with 2-opt heuristic was evaluated across 21 standard instances from the TSPLIB. The results, detailed in Figures 1, 2, and 3, provide a comprehensive overview of the algorithm's effectiveness in terms of solution quality and computational time.

Figure 1 presents a direct comparison between the tour costs obtained by our heuristic (blue) and the known optimal values (green). It is visually evident that the heuristic consistently finds solutions very close to the optimal ones. For the majority of instances, the "Custo Obtido" bar is almost indistinguishable from the "Custo Ótimo" bar, demonstrating the high quality of the solutions found.

To quantify this proximity, Figure 2 illustrates the percentage gap, which is the relative difference between the obtained and optimal costs. The analysis shows an excellent average performance, with many instances like `pr124`, `pr144`, `pr76`, and `att48` achieving a gap of less than 1%. However, the algorithm clearly struggled with specific instances, most notably `rat195` (8.57% gap) and `rat99` (6.11% gap). This suggests that the problem structure of these "rat" instances may present a challenge for this particular GRASP configuration.

Finally, Figure 3 displays the execution time required for each instance. A clear trend emerges: the computational cost grows considerably with the increase in input size (e.g., the progression from `kroA100` to `kroA200`, or `pr124` to `pr152`). The `rat195` instance, which has 195 vertices, also required one of the longest execution times. This runtime is a direct consequence of GRASP's iterative nature and the $O(n^2)$ complexity of the 2-opt local search performed within each of the 100 iterations.

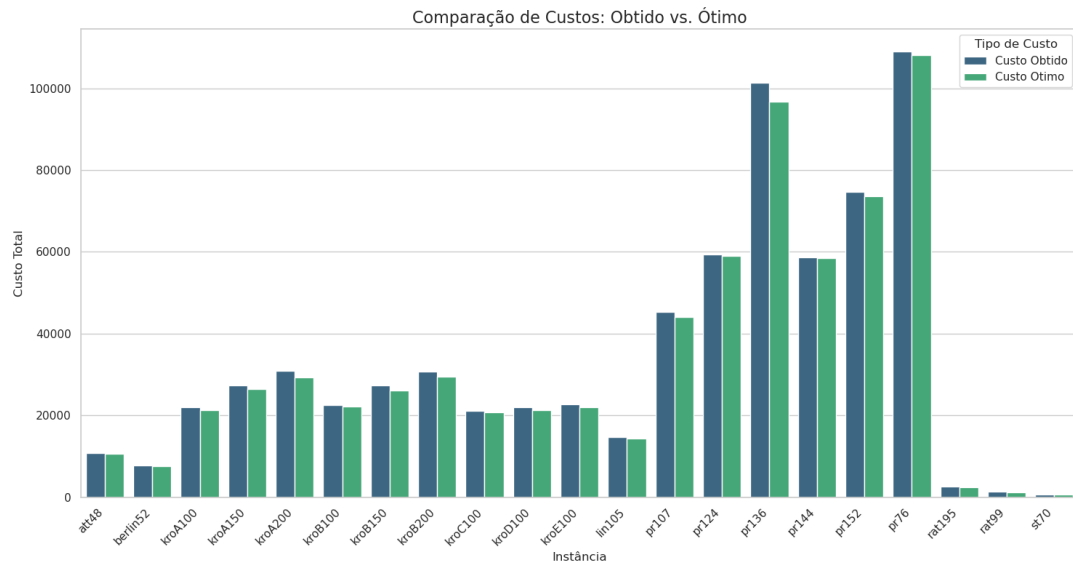


Figure 1: Comparison between the obtained tour cost (blue) and the optimal cost (green) for each instance.

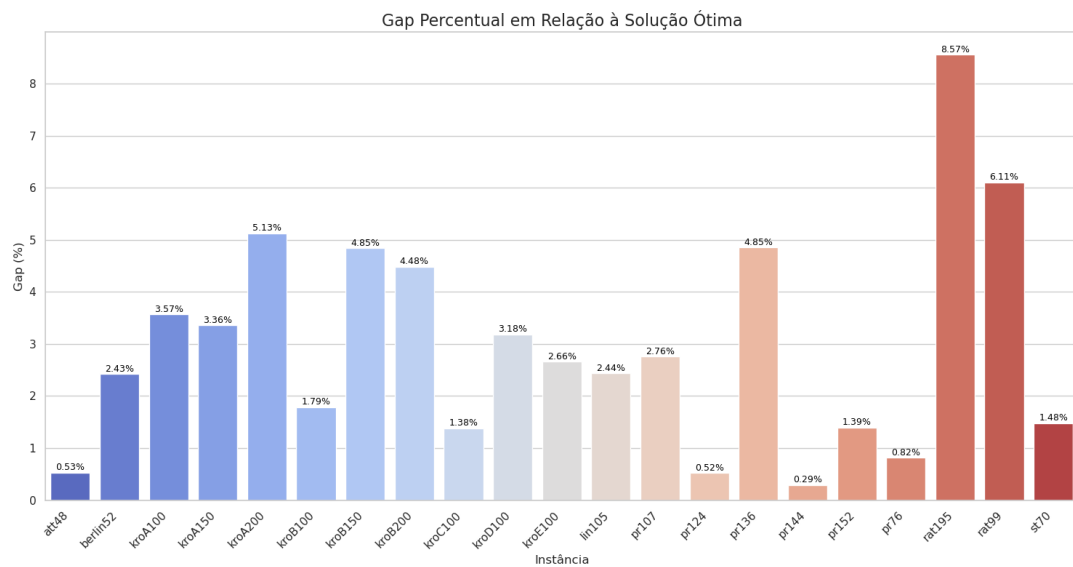


Figure 2: Percentage gap of the obtained solution relative to the optimal solution.

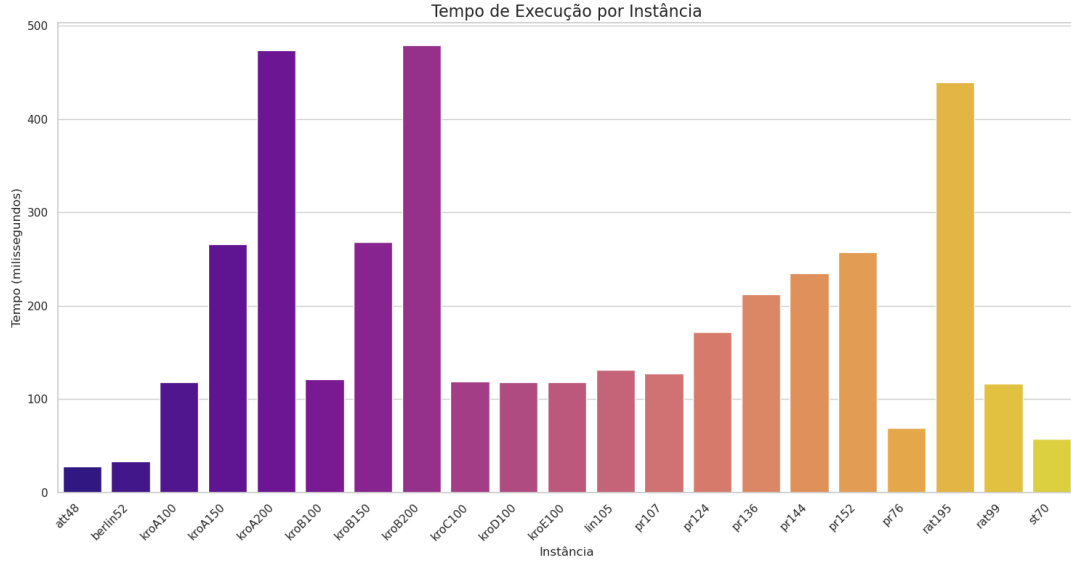


Figure 3: Total execution time in milliseconds (ms) for each instance, representing the total time for all GRASP iterations.

3 Conclusion

In this work, the GRASP metaheuristic, combined with a 2-opt local search, was successfully implemented and evaluated for the Traveling Salesperson Problem. By repeatedly building randomized-greedy solutions and refining them to local optima, the algorithm demonstrated its ability to consistently produce high-quality solutions across a range of standard TSPLIB instances.

The experimental results confirmed that the heuristic achieves a very low average percentage gap relative to the known optimal solutions, establishing it as an effective constructive method. While certain instance types proved challenging (e.g., *rat195*), the tours generated by this approach provide a strong foundation for subsequent improvement phases, serving as excellent initial solutions for more advanced metaheuristics or hybrid algorithms.

References

- [1] VONDOLINGER, N. *algorithms_2_UFMG*. GitHub repository, 2024. Disponível em: https://github.com/nicolasvondolinger/algorithms_2_UFMG. Acesso em: 5 set. 2025.
- [2] UZINSKI, J. *03 Meta-heurísticas: GRASP, Variable neighborhood search (VNS), Meta-heurísticas híbridas, AGCB*. YouTube, 2021. Disponível em: <https://www.youtube.com/watch?v=k-UHm3mfbqk>. Acesso em: 5 set. 2025.